

# Margin Dashboard Exercise

Turn three messy spreadsheets into a tool that answers one question leadership can't currently answer: *did we actually make money on that project?*

|             |                  |
|-------------|------------------|
| TIME BUDGET | 8-12 hours       |
| STACK       | your choice      |
| AI TOOLS    | encouraged       |
| SUBMIT      | git repo + notes |

THE SITUATION

## Context

A digital agency of about forty people runs its entire operation on spreadsheets. Every month, staff log their hours against projects. Salaries sit in a second sheet. What each project was sold for sits in a third.

The data is all there, and it is completely unusable. Nobody can tell you which projects made money, which people are stuck on non-billable work, or what a designer-hour actually costs once you account for the people who never touch a client project.

Your job is to build the tool that answers those questions — a local app that ingests the three spreadsheets and turns them into a dashboard the leadership team could open on a Monday morning.

INPUTS

## The data you'll be given

Three `.xlsx` files, one year of data, in the exact shape the agency actually keeps them. They are real-world messy: header rows are not always in row 1, empty cells are written as -, and dates appear as `May '25`, `January 2026`, or just `January`.

| FILE            | GRAIN                                    | KEY COLUMNS                                                                                                                                                            |
|-----------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Timesheet       | One row per person, per task, per month  | Month · Employee No. · Employee Name · Type of Expense (DL/IDL) · Department · Designation · Category · Ref Code · Project / Task Name · Company · Description · Hours |
| Salary overview | One row per person, one column per month | Employee Name, then January ... December                                                                                                                               |
| Project prices  | One row per project                      | Ref Code · Project Name · Project Price · Sales month · Category · Status                                                                                              |

### Categories

Every timesheet row carries a category. Three of them are billable work charged to a project; the rest are internal time the agency absorbs.

|               |                |           |                    |             |
|---------------|----------------|-----------|--------------------|-------------|
| Projects      | Enhancements   | Hosting   | FC - Meetings      | FC - Leaves |
| FC - Learning | FC - Bug Fixes | FC - Idle | FC - SEO/Marketing | and others  |

The `Ref Code` is what joins a timesheet row to a project price.

Expect the data to be imperfect. Some people have logged hours but no salary row. Some ref codes have hours but no price. How you surface those gaps is part of what we're looking at.

## What to build

We would rather see sixty percent of this scope built well than all of it built badly. Work down the list.

### MUST HAVE

- **Upload.** Take the three spreadsheets through the UI and persist them. Re-uploading a corrected month must not duplicate or destroy the rest of the year.
- **Dashboard.** Total hours, billable hours, cost, revenue and margin for a selected period, with a year and month filter.
- **Project page.** Click any project and get its full picture: price, hours by department, cost, profit, margin, and a per-employee contribution table.
- **Productivity page.** Billable ÷ total hours per employee, filterable by month and year.
- **Category page.** Hours per category, filterable by month and year — where does the time actually go.

### SHOULD HAVE

- **Department drill-down.** Click *Design* and see the hours and cost of every person in it.
- **Per-employee profitability** on each project, using the formula in the next section.
- **Configurable assumptions** — which categories count as billable, and a monthly overhead figure — without editing code.
- **Honest empty and error states.** Missing salary, missing price, a file that isn't what it claims to be.

### STRETCH

- CSV export of any table on screen.
- An employee × category matrix, the pivot the finance team currently builds by hand.
- An audit view showing how each month's cost rates were derived.
- Multi-year support — load 2023, 2024 and 2025 side by side and compare.

## How the numbers work

This is the domain knowledge, not the puzzle — we're handing it to you so you can spend your time on the build. Implement it exactly.

```

direct cost rate / hour    (per person, per month)
    = that month's salary ÷ that month's total logged hours

indirect cost pool        (per month)
    = salaries of people who logged no hours (support staff)
    + everyone else's non-billable time, valued at their direct rate
    + monthly overhead entered by the user

indirect cost rate / hour (per month)
    = indirect cost pool ÷ billable hours that month

employee cost on a project
    = hours × (direct rate + indirect rate)

```

```
employee revenue share
= project price × (employee hours ÷ total project hours)

employee profitability
= (revenue share - employee cost) ÷ revenue share

project profitability
= (project price - total project cost) ÷ project price

productivity
= billable hours ÷ total hours logged
```

#### SELF-CHECK BEFORE YOU SUBMIT

Run a full year with the overhead figure set to zero. Your company-wide total cost should come out equal to total salaries — to the dirham. If it doesn't, something is being counted twice.

## CONSTRAINTS

### Stack, tools and ground rules

#### Pick your own stack

Next.js is a perfectly good default and what most candidates choose. A Python or Node API with any frontend is equally welcome. The only hard requirement: it runs locally on a Mac from a clean checkout with a documented command or two, and it needs no cloud account, no API key and no paid service.

#### Use AI

Use Claude, Cursor, Copilot, whatever you work with — we do. There is no hidden penalty for it and no need to disclose which parts were generated. The one condition is that you own every line: in the follow-up call we will pick a file and ask you to walk us through it, explain a trade-off, and change something live.

#### Respect the time budget

Eight to twelve hours. We are not measuring how many evenings you're willing to give up, and we can tell when a submission took thirty hours. If you run out of time, stop and write down what you'd do next — that note is worth more to us than a rushed extra page.

## DELIVERABLES

### What to send back

- A **git repository** — a link or a zip, with real commit history rather than one commit called “initial”.
- A **README** that gets us running in under five minutes, and states the assumptions you made where the brief was ambiguous.
- **Sample data loaded, or loadable in one click**, so we see a populated dashboard the moment we open it — not an empty shell we have to feed first.
- A **short note** (half a page is plenty): what you'd build next, what you cut and why, and anything you're not happy with.

## How we'll evaluate it

| AREA              | WHAT WE'RE LOOKING FOR                                                                                                     | WEIGHT |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|--------|
| Correctness       | The numbers reconcile. The cost model is internally consistent and doesn't double-count.                                   | 30%    |
| Data handling     | Messy headers, blank cells and odd date formats are absorbed rather than crashing. Re-upload behaviour is thought through. | 20%    |
| Product judgement | The dashboard answers the question without being asked twice. Sensible defaults, useful drill-downs, honest empty states.  | 20%    |
| Code quality      | Readable, organised, the calculation logic separable from the UI. Tests where they earn their keep.                        | 15%    |
| Craft             | It looks like something a director would open willingly. Legible tables, sane typography, no broken layouts.               | 15%    |

We are not scoring the number of features. A submission that nails the must-haves and reconciles to the dirham beats one that ships every stretch goal on numbers we can't trust.

### PLANNING OPTIONAL

## One way to spend the time

Not a requirement — just a shape that tends to work, if it's useful.

|         |                                                                                                                                                                           |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0–2 h   | <b>Read the data</b><br>Open the three files, map the columns, decide your schema and the join keys before writing anything.                                              |
| 2–5 h   | <b>Ingestion and model</b><br>Parsing, persistence, and the calculation layer — with a script that prints the year's totals so you can check them against the self-check. |
| 5–9 h   | <b>The pages</b><br>Dashboard first, then projects, then productivity and categories. Get one page genuinely finished before starting the next.                           |
| 9–11 h  | <b>Edge cases and polish</b><br>Missing salaries, unpriced projects, empty states, the filters. Then make the tables readable.                                            |
| 11–12 h | <b>README and notes</b><br>Setup instructions, your assumptions, and what you'd do next.                                                                                  |

### QUESTIONS

## If something is ambiguous

Ask — a good question is a positive signal, not a negative one. If you'd rather not wait for an answer, make a decision, write it down in your README, and move on. That's the same call you'd make on a Tuesday afternoon here.